Lab01 Report

1. 程式實作方式

實驗 1:

參數設定:

- Batch size: 64, Learning rate: 0.01, Epochs: 10, Activation: ReLU
- Linear Layer:

1.1 Forward: $y = xW + b$

1.2 Backward:

$$\frac{\partial E}{\partial W} = x^T \cdot dy, \quad \frac{\partial E}{\partial b} = \sum dy, \quad \frac{\partial E}{\partial x} = dy \cdot W^T$$

Sigmoid:

- Forward:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Backward:

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} \cdot \frac{d\sigma(x)}{dx}$$

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

Softmax:

- Forward:

$$y_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

減去每一列的最大值 (np.max(x, axis=1)) 以確保數值穩定性

將所有 exponent 結果除以其總和以獲得機率

MLP:

- __init__(): 第一層全連接：輸入 784（28×28），輸出 128

  ReLU 非線性層

  第二層全連接：輸入 128，輸出 10（對應 10 類別）

- Forward:

Input layer → ReLU layer → Ouput layer

- Backward:

傳回 fc2 輸入梯度 → 傳回經 ReLU 的梯度 → 傳回 fc1 輸入梯度

各層梯度分別更新在其內部 (self.W.grad, self.b.grad)

train_one_epoch:

- Forward:

模型輸出進行 Softmax，得到每個類別的機率分佈

減去最大值以防 overflow

- Compute loss: CrossEntropyLoss 計算平均分類損失
- Compute accuracy:

取機率最高的類別作為預測值

與真實標籤比對以計算正確率

- Backward:

由 CrossEntropyLoss 返回 $\frac{\partial E}{\partial y_{pred}}$

- Update parameters:

step()：以 SGD 更新權重

zero_grad()：清除上次的梯度避免累加

evaluate:

- Forward:

先進行 forward，再以 Softmax 轉成機率

- Compute total loss:

累加所有 batch 的損失以計算平均 loss

- Compute correct and total:

統計預測正確的數量與總數

實驗 2:

參數設定:

Batch_size = 128

BasicBlock:

- Input -> [Conv3x3, BN, ReLU] -> [Conv3x3, BN] -> (+) ← Shortcut (Identity 或 1x1 Conv) -> ReLU -> Output
- 當 stride=1 且 in/out channel 相同 → 直接使用 identity (即 self.shortcut = nn.Sequential()，什麼都不做)

否則（需要改變空間大小或通道數）:

- 使用 **1×1 卷積** 改變 channel 數，確保能與 F(x) 相加

ResNet18:

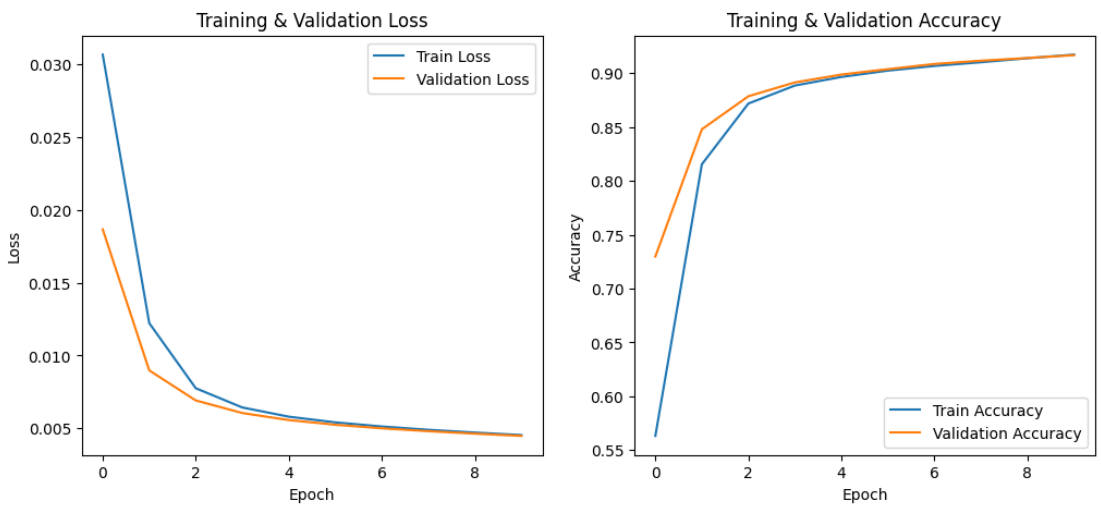
F.avg_pool2d(out, 4)：CIFAR10 的影像是 32×32，經四次 stride=2 後變 4×4，因此用 4×4 池化成 1×1

out.view(out.size(0), -1)：攤平成 (batch_size, 512)

self.fc(out)：輸出分類結果

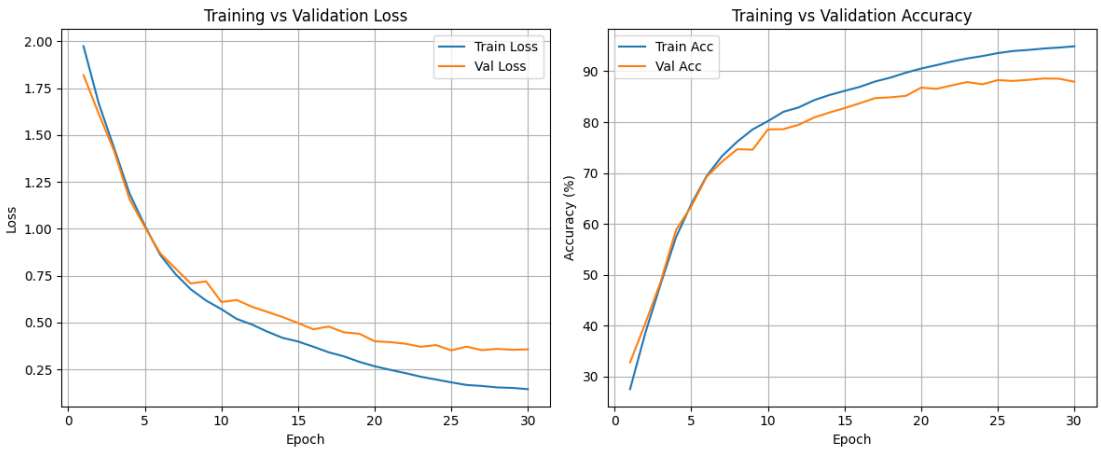
2. 輸出內容

實驗 1:




```
test_loss = 0.004319286697338523, test_acc = 0.9219
```

實驗 2:




```
Best Model Test Accuracy: 90.94%
```

3. 遇到的問題以及解決方法

實驗 1:

P1: 一開始正確率很低，後來發現可能是在 mapping 上面有問題

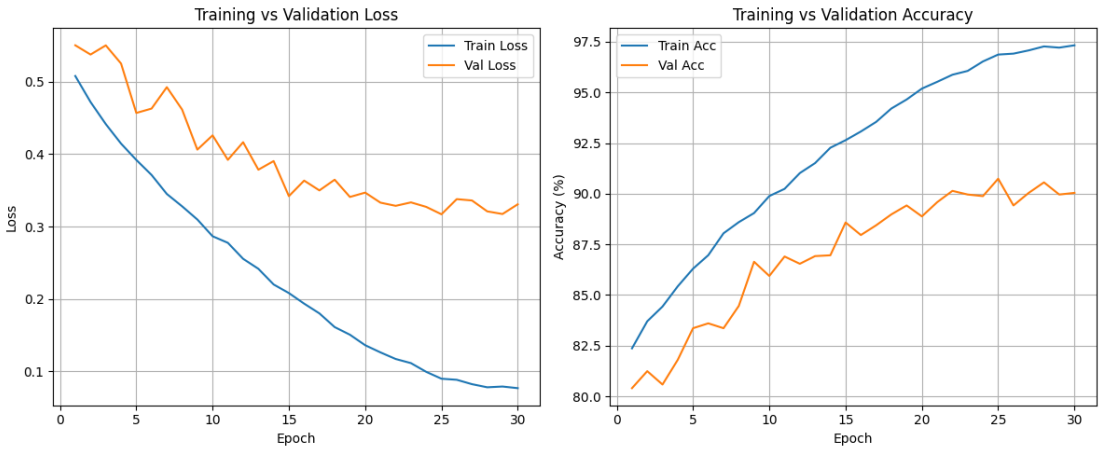
```
def transform(x):
    """map pixels information from range(0, 255) to range(0.01, 1)"""
    #return np.asarray(x) * 0.99 + 0.01 #怪怪的
    x = np.asarray(x) / 255.0 # 0~1
    return x * 0.99 + 0.01 # 0.01~1.0
```

S1: 改掉就正常了

實驗 2:

P1: 一開始有 train_acc 達到 99%以上，但 val_acc 和 test_acc 只有 90%

初，有”過擬合”的狀態出現。



S1:

調整 batch_sizez 和增加 data augmentation，ResNet 架構一開始也寫得

怪怪的，改掉之後就好了